PEMROGRAMAN BERBASIS FRAMEWORK
JOBSHEET 14

NAMA : Muhammad Fahreza Rohmansyah
KELAS : 3E
NO. ABSEN : 15
NIM : 2341720151

PROGRAM STUDI TEKNIK INFORMATIKA
JURUSAN TEKNOLOGI INFORMASI
POLITEKNIK NEGERI MALANG
2026

Link Github:

https://github.com/rezfahreza/PBF_2026.git

Praktikum

1. Install NextAuth

```
PS E:\PBF_2026\JS_14\my-app> npm install next-auth -force
npm warn using --force Recommended protections disabled.

added 15 packages, and audited 486 packages in 34s

158 packages are looking for funding
  run `npm fund` for details

7 vulnerabilities (3 moderate, 4 high)

To address issues that do not require attention, run:
  npm audit fix

To address all issues, run:
  npm audit fix --force

Run `npm audit` for details.
```

2. Buat file /api/auth/[…nextauth].ts dan Konfigurasi API Auth

```
import NextAuth, { NextAuthOptions } from "next-auth"
import CredentialsProvider from "next-auth/providers/credentials"

export const authOptions: NextAuthOptions = {
  session: {
    strategy: "jwt"
  },
  secret: process.env.NEXTAUTH_SECRET,
  providers: [
    CredentialsProvider({
      name: "credentials",
      credentials: {
        fullname: { label: "Full Name", type: "text" },
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        const user : any = {
          id: "1",
          email: credentials?.email,
          password: credentials?.password,
          fullname: credentials?.fullname
        }
        if (user) {
          return user
        } else {
          return null
        }
      }
    })
  ],

  callbacks: {
    async jwt({ token, account, profile, user }:any) {
      if (account?.provider === "credentials" && user) {
        token.email = user.email
      }
      return token
    },
    async session({ session, token }:any) {
      if (token.email) {
        session.user.email = token.email
      }
      return session
    },
  },
};

export default NextAuth(authOptions)
```

3. Tambahkan Secret di env

```
NEXTAUTH_SECRET=d2VsY29tZWJpbGxtdXNpY2NvcHBlcmRhbmdlcmFoZWFkdHJ1bmtmZWF0aGVyc21hbnk=
```

4. Tambahkan SessionProvider pada _app.ts

```
import type { AppProps } from 'next/app';
import AppShell from '@/components/layouts/Appshell';
import Navbar from '@/components/layouts/navbar';
import { SessionProvider } from "next-auth/react";

export default function App({ Component, pageProps : { session, ...pageProps } }: AppProps) {
  return (
    <SessionProvider session={session}>
      <AppShell>
        <Component {...pageProps} />
      </AppShell>
    </SessionProvider>
  )
}
```

5. Tambahkan Tombol Login & Logout

- Modifikasi navbar index.tsx dan navbar.css

```
> my-app > src > components > layouts > navbar > index.tsx > default
import { sign } from 'node:crypto';
import style from './navbar.module.css';
import { signIn } from 'next-auth/react';

const Navbar = () => {
    return (
        <div className={style.navbar}>
            <div className='big'>
                Navbar Component
            </div>
            <button onClick={() => signIn()}>Sign In</button>
        </div>
    );
};

export default Navbar;
```

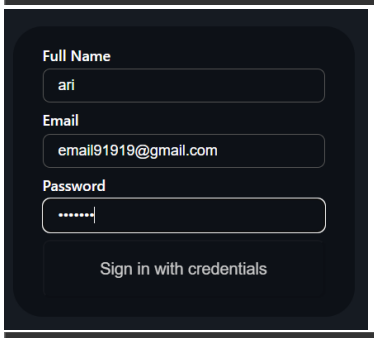
```
.navbar {
  width: 100%;
  height: 60px;
  background-color: #333;
  color: #fff;
  display: flex;
  align-items: center;
  padding: 0 20px;
  justify-content: space-between;
}
```

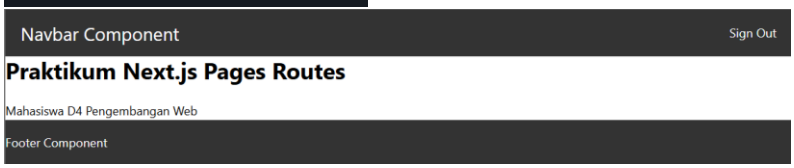
- Uji SingIn

- Modifikasi navbar index.tsx

```
my-app > src > components > layouts > navbar > index.tsx > Navbar
import { sign } from 'node:crypto';
import style from './navbar.module.css';
import { signIn, signOut, useSession } from 'next-auth/react';

const Navbar = () => {
    const { data } = useSession();
    return (
        <div className={style.navbar}>
            <div className='big'>
                Navbar Component
            </div>

            {data ? (
                <button onClick={() => signOut()}>Sign Out</button>
            ) : (
                <button onClick={() => signIn()}>Sign In</button>
            )}

        </div>
    );
};

export default Navbar;
```

- Uji SingOut





6. Menambahkan Data Tambahan

- Modifikasi […nextauth].ts

```
import NextAuth, { NextAuthOptions } from "next-auth"
import CredentialsProvider from "next-auth/providers/credentials"

export const authOptions: NextAuthOptions = {
  session: {
    strategy: "jwt"
  },
  secret: process.env.NEXTAUTH_SECRET,
  providers: [
    CredentialsProvider({
      name: "credentials",
      credentials: {
        fullname: { label: "Full Name", type: "text" },
        email: { label: "Email", type: "email" },
        password: { label: "Password", type: "password" }
      },
      async authorize(credentials) {
        const user : any = {
          id: "1",
          email: credentials?.email,
          password: credentials?.password,
          fullname: credentials?.fullname
        }
        if (user) {
          return user
        } else {
          return null
        }
      }
    })
  ],

  callbacks: {
    async jwt({ token, account, profile, user }:any) {
      if (account?.provider === "credentials" && user) {
        token.email = user.email;
        token.fullname = user.fullname;
      }
      return token
    },
    async session({ session, token }:any) {
      if (token.email) {
        session.user.email = token.email
      }
      if (token.fullname) {
        session.user.name = token.fullname
      }
      return session
    },
  },
};

export default NextAuth(authOptions)
```

- Modifikasi navra.module.scss

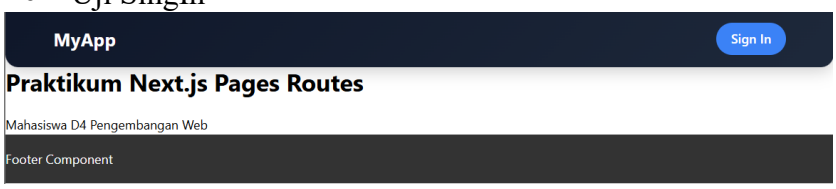
```scss
.navbar {
  width: 100%;
  height: 70px;
  padding: 0 60px;

  display: flex;
  align-items: center;
  justify-content: space-between;

  background: linear-gradient(135deg, #0f172a, #1e293b);
  color: #ffffff;

  box-shadow: 0 4px 20px rgba(0, 0, 0, 0.2);
  border-bottom-left-radius: 16px;
  border-bottom-right-radius: 16px;
}

/* Logo / Brand */
.navbar__brand {
  font-size: 22px;
  font-weight: 700;
  letter-spacing: 0.5px;
}

/* Right Section */
.navbar__right {
  display: flex;
  align-items: center;
  gap: 32px;
}

/* Welcome text */
.navbar__user {
  font-size: 16px;
  font-weight: 500;
  opacity: 0.9;
}
```

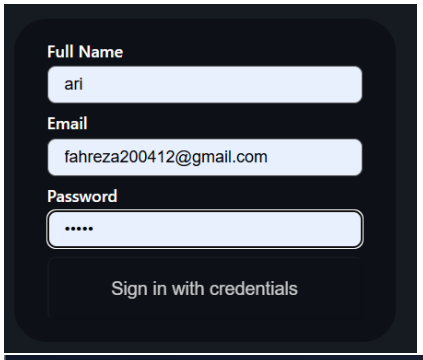
```scss
/* Button */
.navbar__button {
  padding: 8px 18px;
  border-radius: 999px;
  border: none;
  cursor: pointer;
  font-weight: 600;
  transition: all 0.3s ease;
}

/* Sign In */
.navbar__button--primary {
  background-color: #3b82f6;
  color: white;
}

.navbar__button--primary:hover {
  background-color: #2563eb;
  transform: translateY(-2px);
}

/* Sign Out */
.navbar__button--danger {
  background-color: #ef4444;
  color: white;
}

.navbar__button--danger:hover {
  background-color: #dc2626;
  transform: translateY(-2px);
}

/* Responsive */
@media (max-width: 768px) {
  .navbar {
    padding: 0 20px;
  }

  .navbar__brand {
    font-size: 18px;
  }

  .navbar__user {
    display: none;
  }
}
```

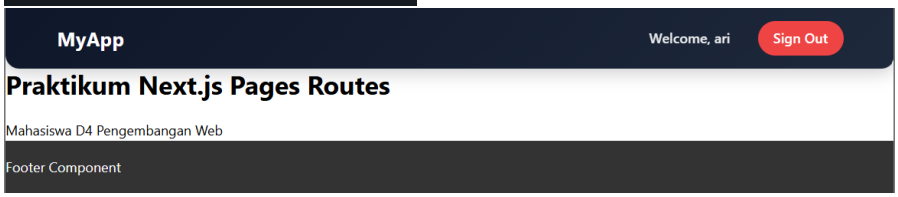
- Modifikasi navbar index.tsx

```
my-app > src > components > layouts > navbar > index.tsx > Navbar
import { sign } from 'node:crypto';
import styles from './navbar.module.css';
import { signIn, signOut, useSession } from 'next-auth/react';
import style from 'styled-jsx/style';

const Navbar = () => {
    const { data } = useSession();
    return (
        <div className={styles.navbar}>
            <div className={styles.navbar__brand}>
                MyApp
            </div>

            <div className={styles.navbar__right}>
                {data ? (
                    <>
                    <div className={styles.navbar__user}>
                        Welcome, {data.user?.name}
                    </div>
                    <button className={`${styles.navbar__button} ${styles["navbar__button--danger"]}`} onClick={()
                        Sign Out
                    </button>
                    </>
                ) : (
                    <button className={`${styles.navbar__button} ${styles["navbar__button--primary"]}`} onClick={(
                        Sign In
                    </button>
                )}
            </div>

        </div>
    );
};

export default Navbar;
```

- Uji SingIn







7. Proteksi Halaman Profile

- Modifikasi file profile index.ts

```
import { useSession } from "next-auth/react";

const HalamanProfile = () => {
    const { data }: { data: any } = useSession();

    return (
        <div>
            <h1>Halaman Profile</h1>
            <h1>Selamat Datang, {data?.user?.fullname}</h1>
        </div>
    );
};

export default HalamanProfile;
```

- Buat file withAuth.ts dan folder dengan nama middleware di src

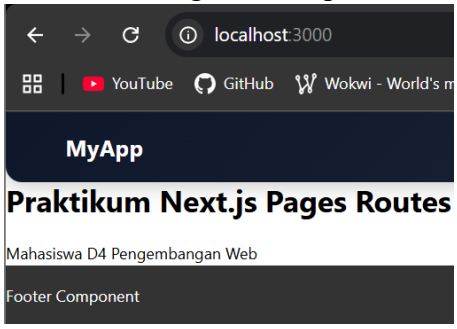
```
> my-app > src > middleware > TS withAuth.ts > withAuth > <function>
import { getToken } from "next-auth/jwt";
import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";

export default function withAuth(
  middleware: NextMiddleware,
  requireAuth: string[] = [],
) {
  return async (req: NextRequest, next: NextFetchEvent) => {
    const pathname = req.nextUrl.pathname;

    if (requireAuth.includes(pathname)) {
      const token = await getToken({
        req,
        secret: process.env.NEXTAUTH_SECRET,
      });

      if (!token) {
        const loginUrl = new URL("/login", req.url);
        return NextResponse.redirect(loginUrl);
      }
    }
    return middleware(req, next);
  }
}
```

- Modifikasi middlewar.ts

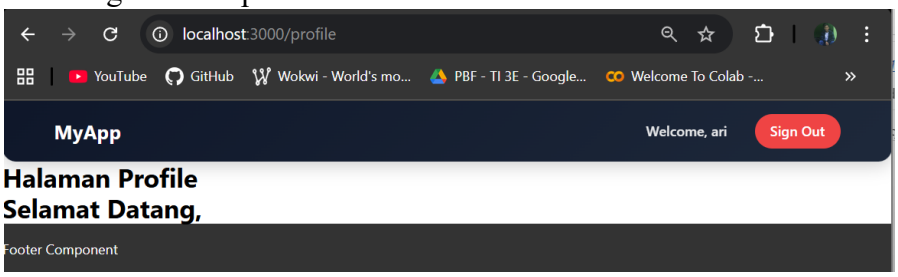
```
import { NextFetchEvent, NextMiddleware, NextRequest, NextResponse } from "next/server";
import withAuth from "./middleware/withAuth";

const baseMiddleware: NextMiddleware = async (
  _req: NextRequest,
  _next: NextFetchEvent,
) => {
  return NextResponse.next();
};

const requireAuthPaths: string[] = [
  "/profile",
  "/produk",
  "/about"
];

export default withAuth(baseMiddleware, requireAuthPaths);
```

8. Pengujian

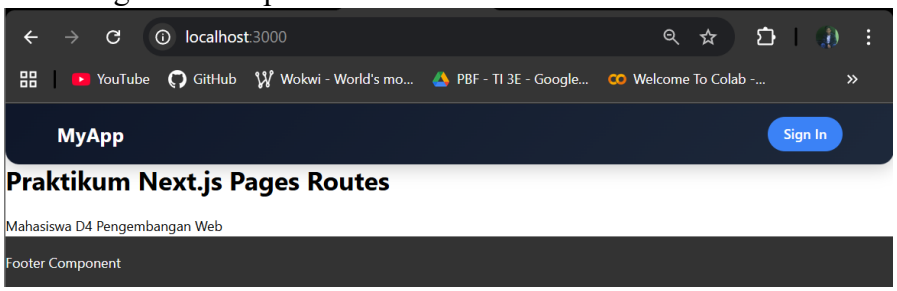
- Belum login akses /profile



- Login akses /profile



- SingOut akses/profile



Tugas Praktikum

1. Implementasikan login menggunakan Credentials Provider.

2. Tambahkan field full name.

3. Tampilkan full name setelah login.

4. Buat halaman profile.

5. Lindungi halaman profile dengan middleware.

Pertanyaan Refleksi

1. Mengapa session menggunakan JWT?

Dalam sistem ini, JWT (JSON Web Token) digunakan sebagai strategi pengelolaan session. Penggunaan JWT memungkinkan session bersifat stateless, di mana data user (seperti email dan fullname) disimpan langsung di dalam token yang terenkripsi. Ini memudahkan verifikasi data tanpa harus selalu memeriksa database di setiap request.

2. Apa perbedaan authorize() dan callback jwt()?

- authorize(): Berfungsi untuk memvalidasi kredensial (seperti email dan password) yang dimasukkan user saat login. Jika valid, ia akan mengembalikan objek user.
- Callback jwt(): Berfungsi untuk memodifikasi atau menambahkan data ke dalam token JWT setelah proses otorisasi berhasil. Di sini, data dari user (seperti fullname) dimasukkan ke dalam token agar bisa diakses nantinya.

3. Mengapa middleware perlu getToken()?

Middleware menggunakan getToken() untuk mengambil dan memverifikasi token JWT dari request yang masuk. Hal ini diperlukan untuk mengecek apakah user memiliki session yang valid (sudah login) sebelum mereka diizinkan mengakses halaman yang dilindungi, seperti /profile.

4. Apa risiko jika NEXTAUTH_SECRET tidak digunakan?

NEXTAUTH_SECRET digunakan untuk mengenkripsi dan menandai (signing) JWT serta mengamankan cookie session. Jika tidak digunakan atau tidak disetel, sistem tidak akan memiliki kunci keamanan untuk memverifikasi keaslian token, sehingga rentan terhadap manipulasi data session oleh pihak luar.

5. Apa perbedaan autentikasi dan otorisasi dalam sistem ini?

- Autentikasi: Proses memverifikasi identitas user (siapa user tersebut), misalnya melalui form login Credentials Provider.
- Otorisasi: Proses mengontrol akses user (apa yang boleh diakses), misalnya menggunakan middleware untuk membatasi akses ke halaman /profile hanya bagi yang sudah login.